

NAME OF THE ASSIGNMENT: -DESCRIPTIVE STATISTICS

Q1. Import the attached CSV files (Diamond.csv) and answer the following questions:

```
import pandas as pd

Diamond=pd.read_csv("Diamond.csv")

Diamond
```

	carat	cut	color	clarity	depth	table	weight	size
price								
0	0.23	Ideal	E	SI2	61.5	55.0	3.95	3.98
326								
1	0.21	Premium	E	SI1	59.8	61.0	3.89	3.84
326								
2	0.23	Good	E	VS1	56.9	65.0	4.05	4.07
327								
3	0.29	Premium	I	VS2	62.4	58.0	4.20	4.23
334								
4	0.31	Good	J	SI2	63.3	58.0	4.34	4.35
335								
...	...	...	...	...	...	...	...	...
..								
53935	0.72	Ideal	D	SI1	60.8	57.0	5.75	5.76
2757								
53936	0.72	Good	D	SI1	63.1	55.0	5.69	5.75
2757								
53937	0.70	Very Good	D	SI1	62.8	60.0	5.66	5.68
2757								
53938	0.86	Premium	H	SI2	61.0	58.0	6.15	6.12
2757								
53939	0.75	Ideal	D	SI2	62.2	55.0	5.83	5.87
2757								

[53940 rows x 9 columns]

A. Create 2 dataframes out of this dataframe – 1 with all numerical variables and other with all categorical variables.

```
df_numerical=Diamond.select_dtypes(include=['int64','float64'])

df_numerical
```

	carat	depth	table	weight	size	price
0	0.23	61.5	55.0	3.95	3.98	326
1	0.21	59.8	61.0	3.89	3.84	326
2	0.23	56.9	65.0	4.05	4.07	327
3	0.29	62.4	58.0	4.20	4.23	334
4	0.31	63.3	58.0	4.34	4.35	335
...	...	...	...	...	...	...
53935	0.72	60.8	57.0	5.75	5.76	2757
53936	0.72	63.1	55.0	5.69	5.75	2757
53937	0.70	62.8	60.0	5.66	5.68	2757
53938	0.86	61.0	58.0	6.15	6.12	2757
53939	0.75	62.2	55.0	5.83	5.87	2757

[53940 rows x 6 columns]

```
df_categorical=Diamond.select_dtypes(include=['object'])
```

```
df_categorical
```

	cut	color	clarity
0	Ideal	E	SI2
1	Premium	E	SI1
2	Good	E	VS1
3	Premium	I	VS2
4	Good	J	SI2
...	...	...	...
53935	Ideal	D	SI1
53936	Good	D	SI1
53937	Very Good	D	SI1
53938	Premium	H	SI2
53939	Ideal	D	SI2

[53940 rows x 3 columns]

B. Calculate the measure of central tendency of numerical variables using Pandas and statistics libraries and check if the calculated values are different between these 2 libraries.

```
import numpy as np
import pandas as pd
import seaborn as sns
import statistics as stat
import matplotlib.pyplot as plt

df_numerical.isnull().sum() # To check the null values
```

carat	0
depth	0
table	0
weight	0
size	0

```
price      0
dtype: int64
```

1. Mean, Median and the Mode are the measure of the Central Tendency

#1.1(a) Mean Calculated through Pandas Library

```
df_numerical.mean()
```

```
carat      0.797940
depth      61.749405
table      57.457184
weight      5.731157
size       5.734526
price     3932.799722
dtype: float64
```

#1.1(b) Mean Calculated through statistics Library

```
round(stat.mean(df_numerical['carat']),6)
```

```
0.79794
```

```
round(stat.mean(df_numerical['depth']),6)
```

```
61.749405
```

```
round(stat.mean(df_numerical['table']),6)
```

```
57.457184
```

```
round(stat.mean(df_numerical['weight']),6)
```

```
5.731157
```

```
round(stat.mean(df_numerical['size']),6)
```

```
5.734526
```

```
round(stat.mean(df_numerical['price']),6)
```

```
3932.799722
```

```
print("Mean Values of numerical variables are matching in both the
cases from Pandas as well as from Statistics")
```

```
Mean Values of numerical variables are matching in both the cases from
Pandas as well as from Statistics
```

#1.2(b) Median Calculated through Pandas Library

```
df_numerical.median()
```

```
carat      0.70
depth      61.80
table      57.00
weight      5.70
size       5.71
price     2401.00
dtype: float64
```

#1.2(b) Median Calculated through statistics Library

```
stat.median(df_numerical['carat'])
```

```
0.7
```

```
stat.median(df_numerical['depth'])
```

```
61.8
```

```
stat.median(df_numerical['table'])
```

```
57.0
```

```
stat.median(df_numerical['weight'])
```

```
5.7
```

```
stat.median(df_numerical['size'])
```

```
5.71
```

```
stat.median(df_numerical['price'])
```

```
2401.0
```

```
print("Median Values of numerical variables are matching in both the  
cases from Pandas as well as from Statistics")
```

```
Median Values of numerical variables are matching in both the cases  
from Pandas as well as from Statistics
```

#1.2(c) Mode Calculated through Pandas Library

```
df_numerical.mode()
```

```
   carat  depth  table  weight  size  price
0    0.3   62.0   56.0    4.37  4.34   605
```

#1.2(c) Mode Calculated through statistics Library

```
stat.mode(df_numerical['carat'])
```

```
0.3
```

```
stat.mode(df_numerical['depth'])
```

```

62.0
stat.mode(df_numerical['table'])

56.0
stat.mode(df_numerical['weight'])

4.37
stat.mode(df_numerical['size'])

4.34
stat.mode(df_numerical['price'])

605

print("Mode Values of numerical variables are matching in both the
cases from Pandas as well as from Statistics")

Mode Values of numerical variables are matching in both the cases from
Pandas as well as from Statistics

```

C. Check the skewness of all numeric variables. Mention against each variable if its highly skewed/light skewed/ Moderately skewed.

```

skewness_numerical_variable =
df_numerical[['carat', 'depth', 'table', 'weight', 'size', 'price']].skew()

skewness_numerical_variable

carat      1.116646
depth     -0.082294
table      0.796896
weight     0.378676
size       2.434167
price      1.618395
dtype: float64

def skewness_level(skewness_numerical_variable):
    if skewness_numerical_variable > 1 or skewness_numerical_variable
< -1:
        return "Highly Skewed"
    elif 0.5 < abs(skewness_numerical_variable) <= 1:
        return "Moderately Skewed"
    else:
        return "Lightly Skewed"
skewness_levels = skewness_numerical_variable.apply(skewness_level)

```

```
print(skewness_levels)

carat      Highly Skewed
depth      Lightly Skewed
table      Moderately Skewed
weight     Lightly Skewed
size       Highly Skewed
price      Highly Skewed
dtype: object
```

D. Use the different transformation techniques to convert skewed data found in previous question into normal distribution.

```
from scipy.stats import boxcox

# Log Transformation
df_numerical['log_carat'] = np.log(df_numerical['carat'] + 1) #
Adding 1 to avoid log(0)

# Square Root Transformation
df_numerical['sqrt_carat'] = np.sqrt(df_numerical['carat'])

# Cube Root Transformation
df_numerical['cbrt_carat'] = np.cbrt(df_numerical['carat'])

# Box-Cox Transformation
df_numerical['boxcox_carat'], _ = boxcox(df_numerical['carat'] + 1) #
Adding 1 to avoid log(0)

# Log Transformation
df_numerical['log_depth'] = np.log(df_numerical['depth']+1) # Adding
1 to avoid log(0)

# Square Root Transformation
df_numerical['sqrt_depth'] = np.sqrt(df_numerical['depth'])

# Cube Root Transformation
df_numerical['cbrt_depth'] = np.cbrt(df_numerical['depth'])

# Box-Cox Transformation
df_numerical['boxcox_depth'], _ = boxcox(df_numerical['depth']+1) #
Adding 1 to avoid log(0)

# Log Transformation
df_numerical['log_table'] = np.log(df_numerical['table'] + 1) #
Adding 1 to avoid log(0)

# Square Root Transformation
df_numerical['sqrt_table'] = np.sqrt(df_numerical['table'])
```

```

# Cube Root Transformation
df_numerical['cbrt_table'] = np.cbrt(df_numerical['table'])

# Box-Cox Transformation
df_numerical['boxcox_table'], _ = boxcox(df_numerical['table'] + 1) #
Adding 1 to avoid log(0)

# Log Transformation
df_numerical['log_weight'] = np.log(df_numerical['weight'] + 1) #
Adding 1 to avoid log(0)

# Square Root Transformation
df_numerical['sqrt_weight'] = np.sqrt(df_numerical['weight'])

# Cube Root Transformation
df_numerical['cbrt_weight'] = np.cbrt(df_numerical['weight'])

# Box-Cox Transformation
df_numerical['boxcox_weight'], _ = boxcox(df_numerical['weight'] + 1)
# Adding 1 to avoid log(0)

# Log Transformation
df_numerical['log_size'] = np.log(df_numerical['size'] + 1) # Adding
1 to avoid log(0)

# Square Root Transformation
df_numerical['sqrt_size'] = np.sqrt(df_numerical['carat'])

# Cube Root Transformation
df_numerical['cbrt_size'] = np.cbrt(df_numerical['size'])

# Box-Cox Transformation
df_numerical['boxcox_size'], _ = boxcox(df_numerical['size'] + 1) #
Adding 1 to avoid log(0)

# Log Transformation
df_numerical['log_price'] = np.log(df_numerical['price'] + 1) #
Adding 1 to avoid log(0)

# Square Root Transformation
df_numerical['sqrt_price'] = np.sqrt(df_numerical['price'])

# Cube Root Transformation
df_numerical['cbrt_price'] = np.cbrt(df_numerical['price'])

# Box-Cox Transformation
df_numerical['boxcox_price'], _ = boxcox(df_numerical['price'] + 1) #
Adding 1 to avoid log(0)

```

```
Transformed_carat_variable =  
df_numerical[['log_carat', 'sqrt_carat', 'cbrt_carat', 'boxcox_carat']].skew()  
Transformed_carat_variable
```

```
log_carat      0.580654  
sqrt_carat     0.548471  
cbrt_carat     0.386589  
boxcox_carat   0.117887  
dtype: float64
```

```
Transformed_depth_variable =  
df_numerical[['log_depth', 'sqrt_depth', 'cbrt_depth', 'boxcox_depth']].skew()  
Transformed_depth_variable
```

```
log_depth      -0.359117  
sqrt_depth     -0.219913  
cbrt_depth     -0.267034  
boxcox_depth   0.068120  
dtype: float64
```

```
Transformed_table_variable =  
df_numerical[['log_table', 'sqrt_table', 'cbrt_table', 'boxcox_table']].skew()  
Transformed_table_variable
```

```
log_table      0.602231  
sqrt_table     0.692227  
cbrt_table     0.660170  
boxcox_table   0.000000  
dtype: float64
```

```
Transformed_weight_variable =  
df_numerical[['log_weight', 'sqrt_weight', 'cbrt_weight', 'boxcox_weight']].skew()  
Transformed_weight_variable
```

```
log_weight     -0.073063  
sqrt_weight     0.088258  
cbrt_weight    -0.322953  
boxcox_weight   0.004618  
dtype: float64
```

```
Transformed_size_variable =  
df_numerical[['log_size', 'sqrt_size', 'cbrt_size', 'boxcox_size']].skew()  
)  
Transformed_size_variable
```

```
log_size       0.006600  
sqrt_size      0.548471
```



```

cbirt_size      -0.132075
boxcox_size     -0.000807
dtype: float64

Transformed_price_variable =
df_numerical[['log_price', 'sqrt_price', 'cbirt_price', 'boxcox_price']].skew()
Transformed_price_variable

log_price       0.115926
sqrt_price      0.844396
cbirt_price     0.591189
boxcox_price    0.025726
dtype: float64

```

E. Create a user defined function in python to check the outliers using IQR method. Then pass all numeric variables in that function to check outliers.

```

def detect_outliers_iqr(df_numerical):

    outliers = pd.DataFrame(index=df_numerical.index,
                             columns=df_numerical.columns)

    for col in df_numerical.columns:
        Q1 = df_numerical[col].quantile(0.25)
        Q3 = df_numerical[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers[col] = (df_numerical[col] < lower_bound) |
        (df_numerical[col] > upper_bound)

    return outliers

outliers_df = detect_outliers_iqr(df_numerical)
print(outliers_df)

```

	carat	depth	table	weight	size	price
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	True	True	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
...	...	...	...	...	...	...
53935	False	False	False	False	False	False

53936	False	False	False	False	False	False
53937	False	False	False	False	False	False
53938	False	False	False	False	False	False
53939	False	False	False	False	False	False

[53940 rows x 6 columns]

F. Convert categorical variables into numerical variables using LabelEncoder technique.

```
df_categorical
```

	cut	color	clarity
0	Ideal	E	SI2
1	Premium	E	SI1
2	Good	E	VS1
3	Premium	I	VS2
4	Good	J	SI2
...	...	...	...
53935	Ideal	D	SI1
53936	Good	D	SI1
53937	Very Good	D	SI1
53938	Premium	H	SI2
53939	Ideal	D	SI2

[53940 rows x 3 columns]

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
le.fit(df_categorical['cut'])
```

```
LabelEncoder()
```

```
list(le.classes_)
```

```
['Fair', 'Good', 'Ideal', 'Premium', 'Very Good']
```

```
transform_cut=le.transform(df_categorical['cut'])
transform_cut
```

```
array([2, 3, 1, ..., 4, 3, 2])
```

```
le.fit(df_categorical['color'])
```

```
LabelEncoder()
```

```
list(le.classes_)
```

```
['D', 'E', 'F', 'G', 'H', 'I', 'J']
```

```

transform_color=le.transform(df_categorical['color'])
transform_color

array([1, 1, 1, ..., 0, 4, 0])

le.fit(df_categorical['clarity'])
LabelEncoder()

list(le.classes_)

['I1', 'IF', 'SI1', 'SI2', 'VS1', 'VS2', 'VVS1', 'VVS2']

transform_clarity=le.transform(df_categorical['clarity'])
transform_clarity

array([3, 2, 4, ..., 2, 3, 3])

```

G. Use both the feature scaling techniques (standardscaler/min max scaler) on all the variables.

```

# Using standard scalar technique on numerical variables
df_standard_numerical = (df_numerical -
df_numerical.mean())/df_numerical.std()
df_standard_numerical.head()

```

	carat	depth	table	weight	size	price
0	-1.198157	-0.174090	-1.099662	-1.587823	-1.536181	-0.904087
1	-1.240350	-1.360726	1.585514	-1.641310	-1.658759	-0.904087
2	-1.198157	-3.384987	3.375631	-1.498677	-1.457382	-0.903836
3	-1.071577	0.454129	0.242926	-1.364959	-1.317293	-0.902081
4	-1.029384	1.082348	0.242926	-1.240155	-1.212227	-0.901831

```

# Using MinMax scalar technique on numerical variables

MinMax_Scalar=(df_numerical-(df_numerical.min()))/
((df_numerical.max())-(df_numerical.min()))
MinMax_Scalar.head()

```

	carat	depth	table	weight	size	price
0	0.006237	0.513889	0.230769	0.367784	0.067572	0.000000
1	0.002079	0.466667	0.346154	0.362197	0.065195	0.000000
2	0.006237	0.386111	0.423077	0.377095	0.069100	0.000054
3	0.018711	0.538889	0.288462	0.391061	0.071817	0.000433
4	0.022869	0.563889	0.288462	0.404097	0.073854	0.000487

H. Create the Histogram for all numeric variables and draw the KDE plot on that.

```

plt.figure(figsize=(18, 12))

```

```

# Original Distribution
plt.subplot(3, 2, 1)
sns.histplot(data=df_numerical['carat'], kde=True)
plt.title('Original carat')

# Log Transformation
plt.subplot(3, 2, 2)
sns.histplot(data=df_numerical['log_carat'], kde=True)
plt.title('Log Transformed carat')

# Square Root Transformation
plt.subplot(3, 2, 3)
sns.histplot(data=df_numerical['sqrt_carat'], kde=True)
plt.title('Square Root Transformed carat')

# Cube Root Transformation
plt.subplot(3, 2, 4)
sns.histplot(data=df_numerical['cbrt_carat'], kde=True)
plt.title('Cube Root Transformed carat')

# Box-Cox Transformation
plt.subplot(3, 2, 5)
sns.histplot(data=df_numerical['boxcox_carat'], kde=True)
plt.title('Box-Cox Transformed carat')

plt.tight_layout()
plt.show()

```

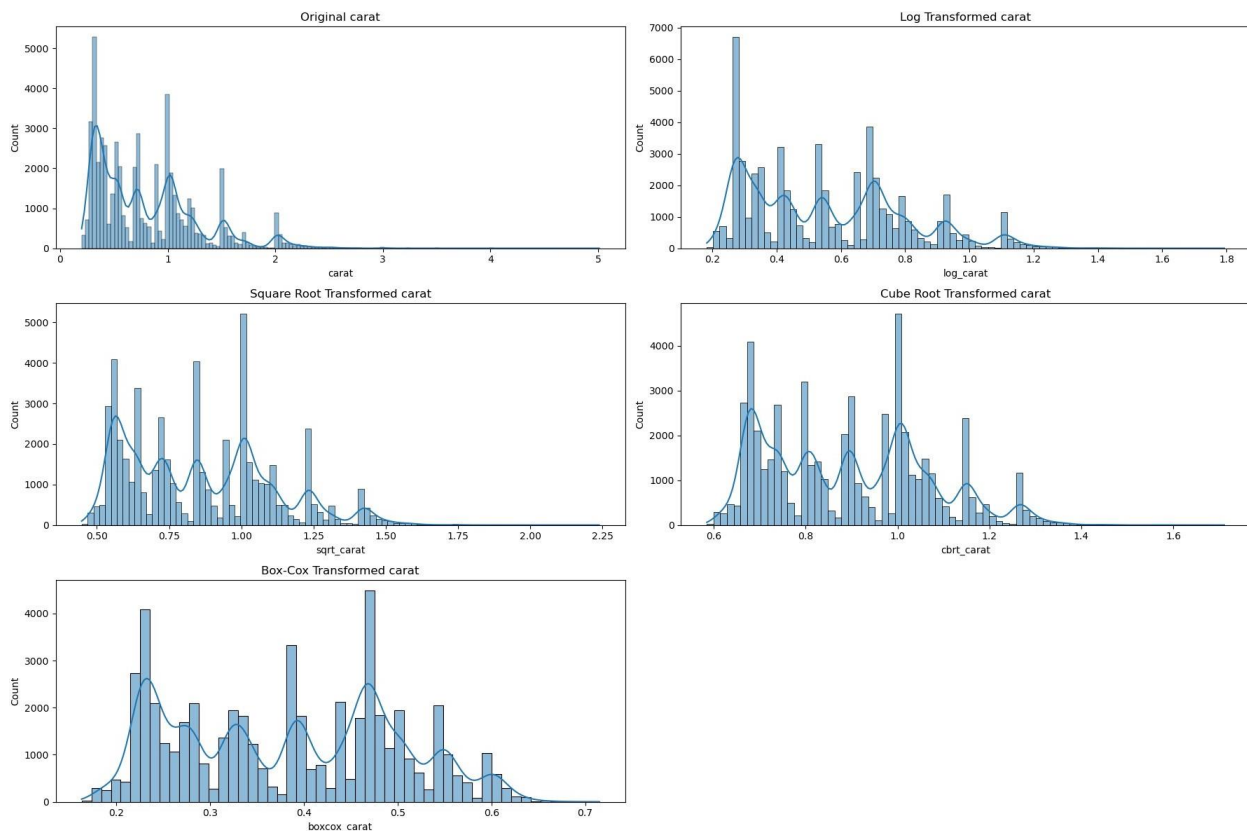
```

C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
  with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
  with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
  with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
  with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed

```

in a future version. Convert inf values to NaN before operating instead.

```
with pd.option_context('mode.use_inf_as_na', True):
```



```
plt.figure(figsize=(18, 12))

# Original Distribution
plt.subplot(3, 2, 1)
sns.histplot(data=df_numerical['depth'], kde=True)
plt.title('Original depth')

# Log Transformation
plt.subplot(3, 2, 2)
sns.histplot(data=df_numerical['log_depth'], kde=True)
plt.title('Log Transformed depth')

# Square Root Transformation
plt.subplot(3, 2, 3)
sns.histplot(data=df_numerical['sqrt_depth'], kde=True)
plt.title('Square Root Transformed depth')

# Cube Root Transformation
plt.subplot(3, 2, 4)
sns.histplot(data=df_numerical['cbrt_depth'], kde=True)
```

```
plt.title('Cube Root Transformed depth')
```

```
# Box-Cox Transformation
```

```
plt.subplot(3, 2, 5)
```

```
sns.histplot(data=df_numerical['boxcox_depth'], kde=True)
```

```
plt.title('Box-Cox Transformed depth')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

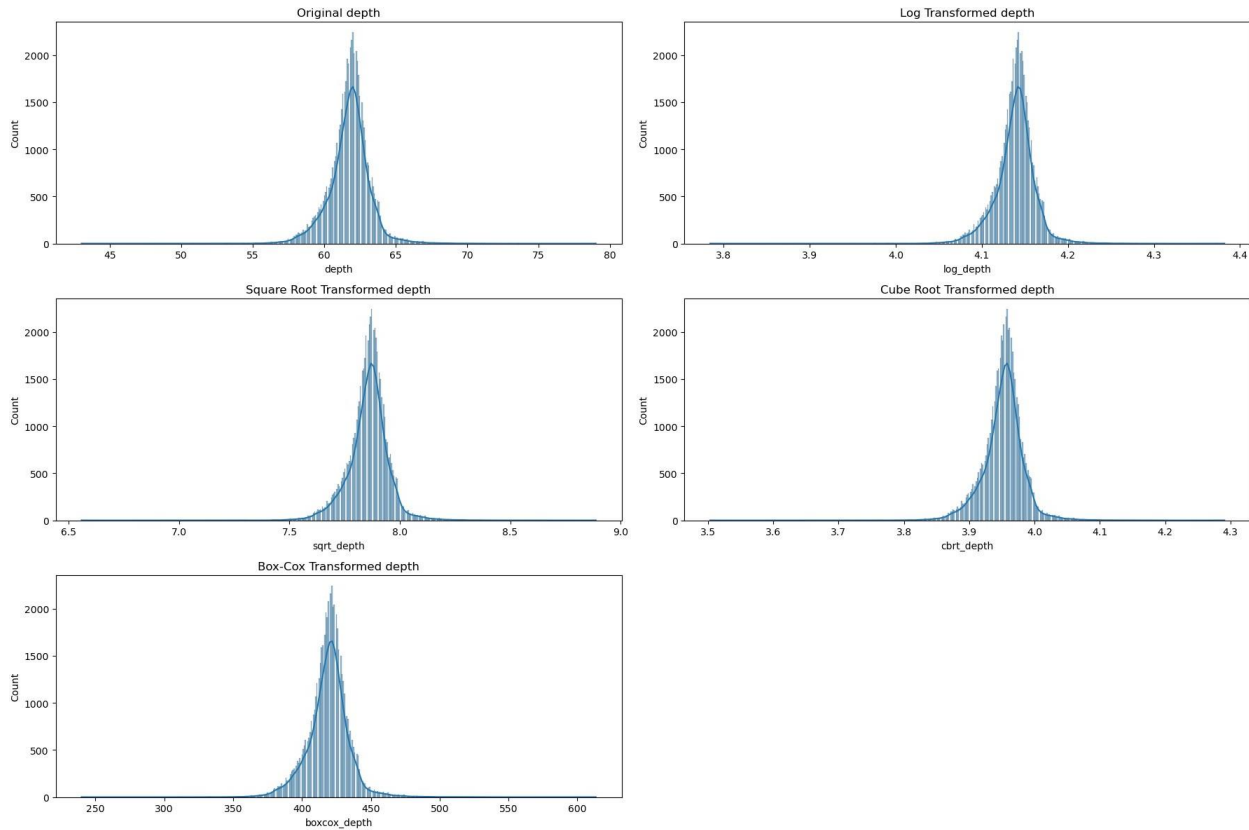
```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```



```
plt.figure(figsize=(18, 12))

# Original Distribution
plt.subplot(3, 2, 1)
sns.histplot(data=df_numerical['table'], kde=True)
plt.title('Original table')

# Log Transformation
plt.subplot(3, 2, 2)
sns.histplot(data=df_numerical['log_table'], kde=True)
plt.title('Log Transformed table')

# Square Root Transformation
plt.subplot(3, 2, 3)
sns.histplot(data=df_numerical['sqrt_table'], kde=True)
plt.title('Square Root Transformed table')

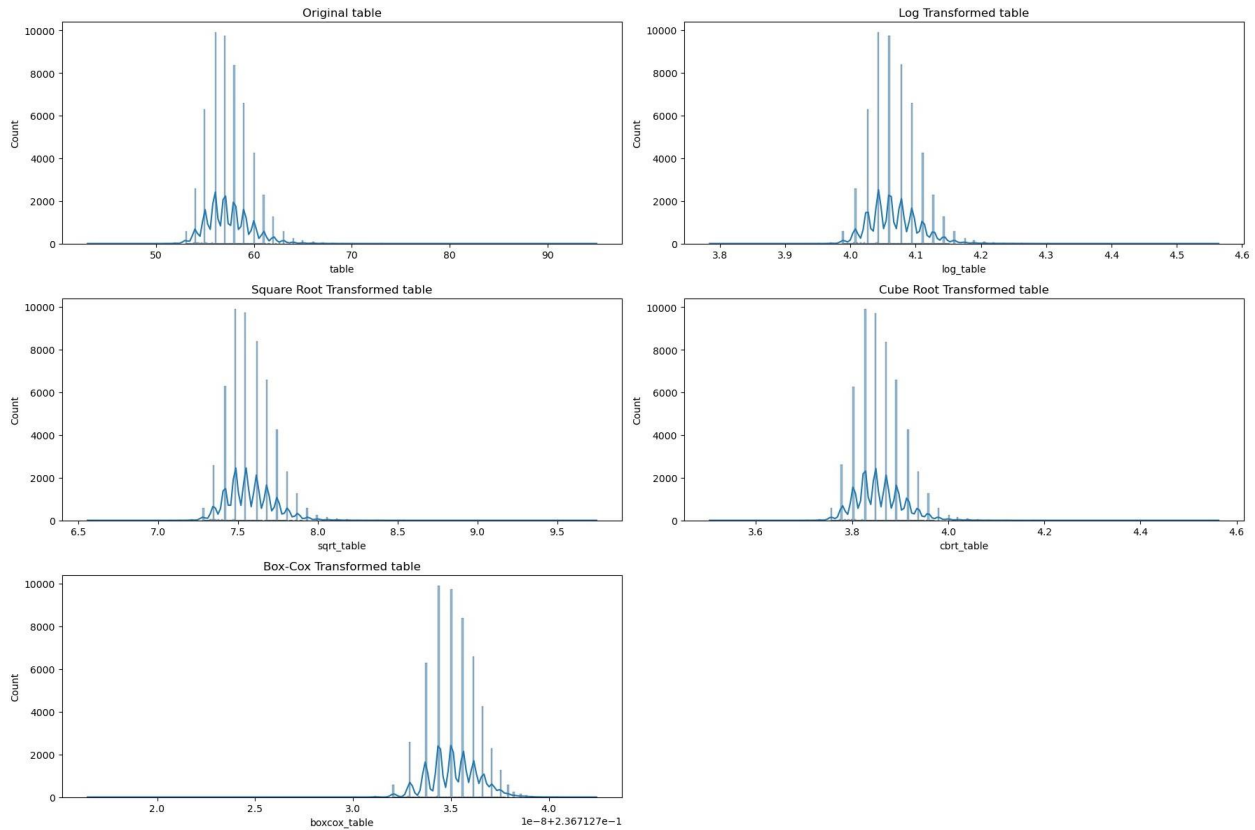
# Cube Root Transformation
plt.subplot(3, 2, 4)
sns.histplot(data=df_numerical['cbrt_table'], kde=True)
plt.title('Cube Root Transformed table')

# Box-Cox Transformation
plt.subplot(3, 2, 5)
sns.histplot(data=df_numerical['boxcox_table'], kde=True)
```

```
plt.title('Box-Cox Transformed table')

plt.tight_layout()
plt.show()

C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
```

```
plt.figure(figsize=(18, 12))

# Original Distribution
plt.subplot(3, 2, 1)
sns.histplot(data=df_numerical['weight'], kde=True)
plt.title('Original weight')

# Log Transformation
plt.subplot(3, 2, 2)
sns.histplot(data=df_numerical['log_weight'], kde=True)
plt.title('Log Transformed weight')

# Square Root Transformation
plt.subplot(3, 2, 3)
sns.histplot(data=df_numerical['sqrt_weight'], kde=True)
plt.title('Square Root Transformed weight')

# Cube Root Transformation
plt.subplot(3, 2, 4)
sns.histplot(data=df_numerical['cbt_weight'], kde=True)
plt.title('Cube Root Transformed weight')

# Box-Cox Transformation
plt.subplot(3, 2, 5)
sns.histplot(data=df_numerical['boxcox_weight'], kde=True)
```

```
plt.title('Box-Cox Transformed weight')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

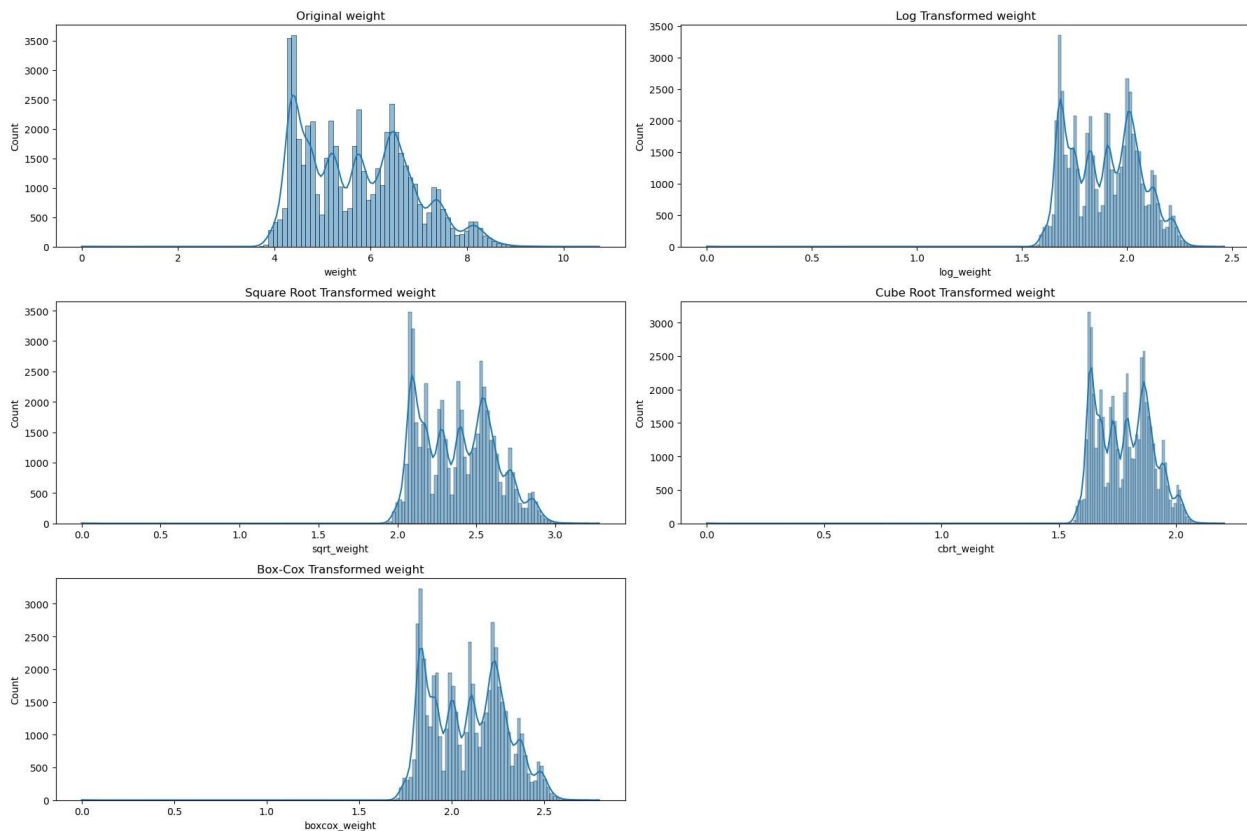
```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```

```
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
```

```
FutureWarning: use_inf_as_na option is deprecated and will be removed  
in a future version. Convert inf values to NaN before operating  
instead.
```

```
    with pd.option_context('mode.use_inf_as_na', True):
```



```
plt.figure(figsize=(18, 12))

# Original Distribution
plt.subplot(3, 2, 1)
sns.histplot(data=df_numerical['size'], kde=True)
plt.title('Original size')

# Log Transformation
plt.subplot(3, 2, 2)
sns.histplot(data=df_numerical['log_size'], kde=True)
plt.title('Log Transformed size')

# Square Root Transformation
plt.subplot(3, 2, 3)
sns.histplot(data=df_numerical['sqrt_size'], kde=True)
plt.title('Square Root Transformed size')

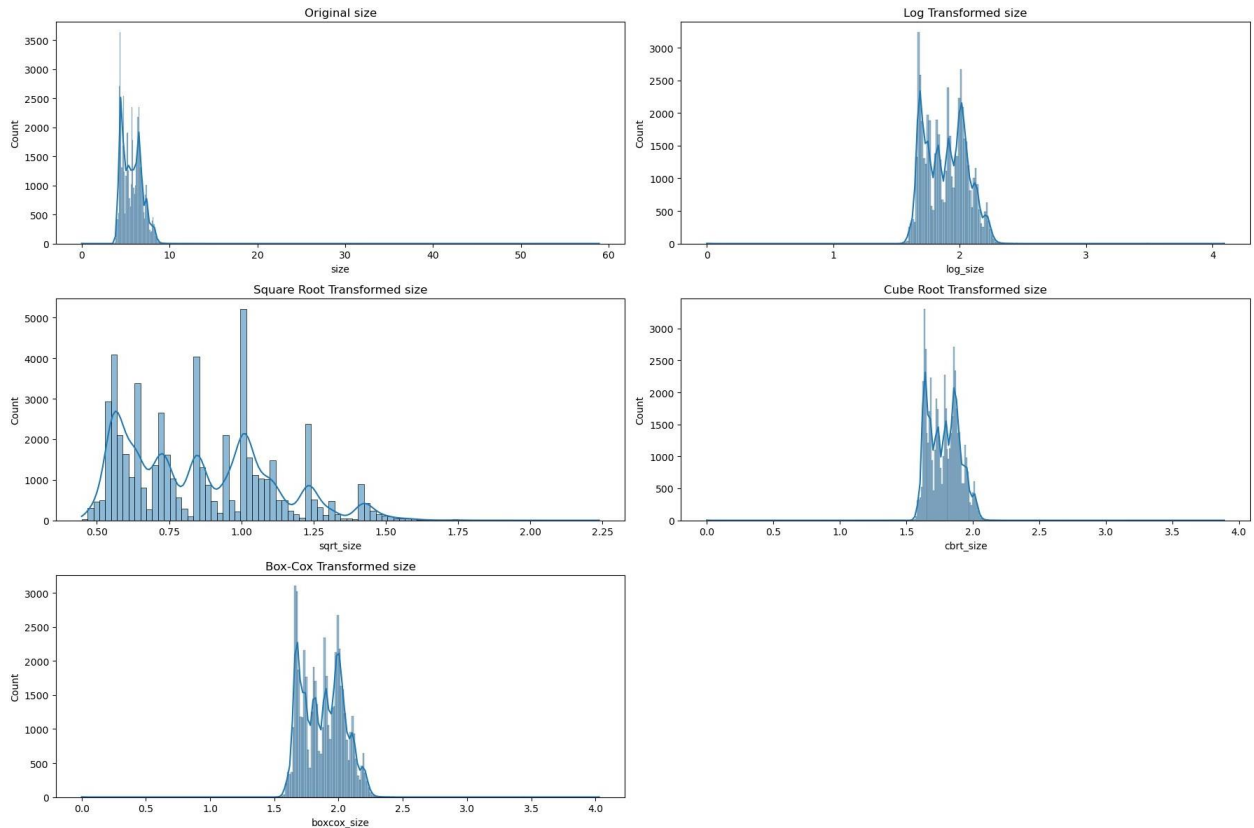
# Cube Root Transformation
plt.subplot(3, 2, 4)
sns.histplot(data=df_numerical['cbrt_size'], kde=True)
plt.title('Cube Root Transformed size')

# Box-Cox Transformation
plt.subplot(3, 2, 5)
sns.histplot(data=df_numerical['boxcox_size'], kde=True)
```

```
plt.title('Box-Cox Transformed size')

plt.tight_layout()
plt.show()

C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
```



```
plt.figure(figsize=(18, 12))

# Original Distribution
plt.subplot(3, 2, 1)
sns.histplot(data=df_numerical['price'], kde=True)
plt.title('Original price')

# Log Transformation
plt.subplot(3, 2, 2)
sns.histplot(data=df_numerical['log_price'], kde=True)
plt.title('Log Transformed price')

# Square Root Transformation
plt.subplot(3, 2, 3)
sns.histplot(data=df_numerical['sqrt_price'], kde=True)
plt.title('Square Root Transformed price')

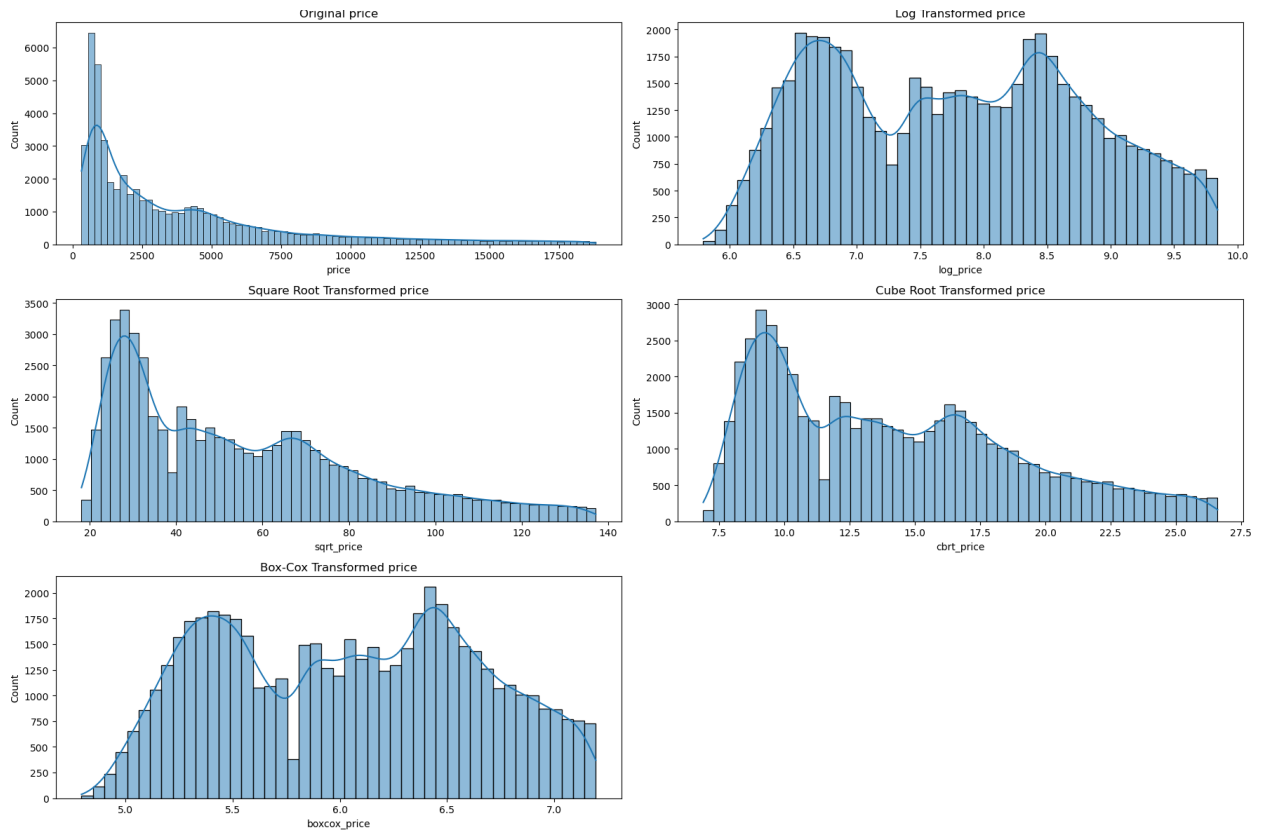
# Cube Root Transformation
plt.subplot(3, 2, 4)
sns.histplot(data=df_numerical['cbqrt_price'], kde=True)
plt.title('Cube Root Transformed price')

# Box-Cox Transformation
plt.subplot(3, 2, 5)
sns.histplot(data=df_numerical['boxcox_price'], kde=True)
```

```
plt.title('Box-Cox Transformed price')

plt.tight_layout()
plt.show()

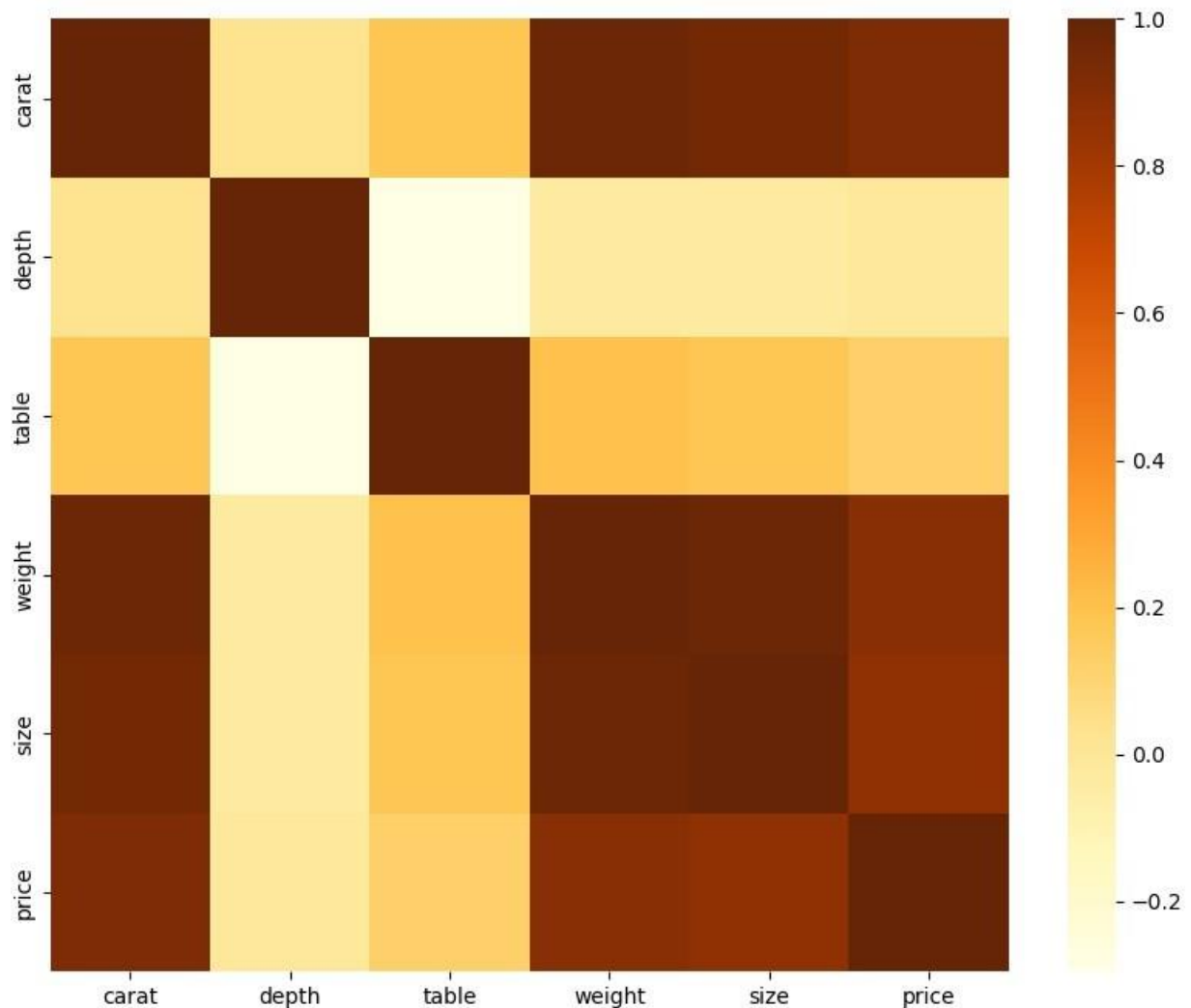
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Ananconda4\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
    with pd.option_context('mode.use_inf_as_na', True):
```



I. Check the correlation between all the numeric variables using HeatMap and try to draw some conclusion about the data.

```
plt.figure(figsize=(10,8))
sns.heatmap(df_numerical.corr(),cmap='YlOrBr')
```

<Axes: >



Above data depicts that there is positive coorelation between carat,price,size and weight

1. When the carat is increasing the price is increasing
2. When the size is increasing,the wieght is also increasing and accordingly the price is also increasing.
3. When the carat is increasing size,weight and price is also increasing.

Q2. Explain Gradient descent in detail. How changing the values of learning rate can impact the convergence in Gradient Descent.

Gradient Descent: A Detailed Explanation

Gradient means slope. Gradient descent means descending a slope to reach lowest point on that surface.

Objective

Gradient Descent is an iterative algorithm, that starts from a random point on a function and travels down its slope in steps until it reaches the lowest point of that function. The objective of the Gradient descent algorithm is to find out the value of "x" such that "y" is minimum.

1. Understanding the Loss Function

- The **Loss function** measures how well the model's predictions align with the actual data. For example, in linear regression, the loss function is often the Mean Squared Error (MSE), which calculates the average squared difference between the predicted values and the actual values.
- The goal of Gradient Descent is to find the model parameters (e.g., weights and biases) that minimize this loss function.

2. The Gradient Descent Algorithm

The Gradient Descent algorithm involves the following steps:

1. **Step-1:** Find the slope (differentiate "y" with respect to "x") of the objective function with respect to 'x'.
2. **Step-2:** Randomly initialize the Objective function.
3. **Step-3:** Putting the value of starting points in Gradient.
4. **Step-4:** Find the step size by the multiplication of step Size= (Gradient) x (Learning rate).
5. **Step-5:** Find the new value of x. New Value (of x)= old value (of x)- (step size) or use the formula as New (x)= Old(x)- $\eta \times \frac{dy}{dx}$. Where η is the learning rate.
6. **Repeat:** Continue iterating steps 3,4 and 5 until the parameters no longer change significantly, or the loss function is minimized.

Impact of Learning Rate on Convergence

The **learning rate** η is a crucial hyperparameter in Gradient Descent. It controls the step size of the parameter updates. The choice of learning rate can significantly affect the efficiency and effectiveness of the convergence process.

1. Small Learning Rate

- **Slow Convergence:** If the learning rate is too small, the algorithm will take tiny steps towards the minimum of the loss function. While this ensures that the algorithm moves steadily towards convergence, it also means that the process can be extremely slow, requiring many iterations.
- **Risk of Getting Stuck in Local Minima:** A very small learning rate can also cause the algorithm to get stuck in local minima, especially in more complex loss functions that are not convex.

2. Large Learning Rate

- **Overshooting:** If the learning rate is too large, the algorithm might take steps that are too big. Instead of moving closer to the minimum, it might overshoot it, causing the loss function to oscillate or even diverge. This means that the algorithm may fail to converge altogether.
- **Faster Convergence (if not too large):** If the learning rate is appropriately large, it can speed up the convergence process by taking larger steps towards the minimum, reducing the number of iterations needed.

3. Learning Rate Scheduling

- **Adaptive Learning Rates:** In practice, it can be beneficial to start with a larger learning rate and gradually reduce it as the algorithm progresses. This approach helps the algorithm take larger steps in the beginning and fine-tune the parameters with smaller steps as it nears convergence. Techniques like **learning rate annealing** or **learning rate decay** implement this strategy.

Conclusion

Gradient Descent is a powerful optimization technique, but the learning rate is a critical parameter that must be chosen carefully. Too small a learning rate results in slow convergence, while too large a learning rate risks overshooting and divergence. Understanding and tuning the learning rate is essential for efficient and effective model training.

